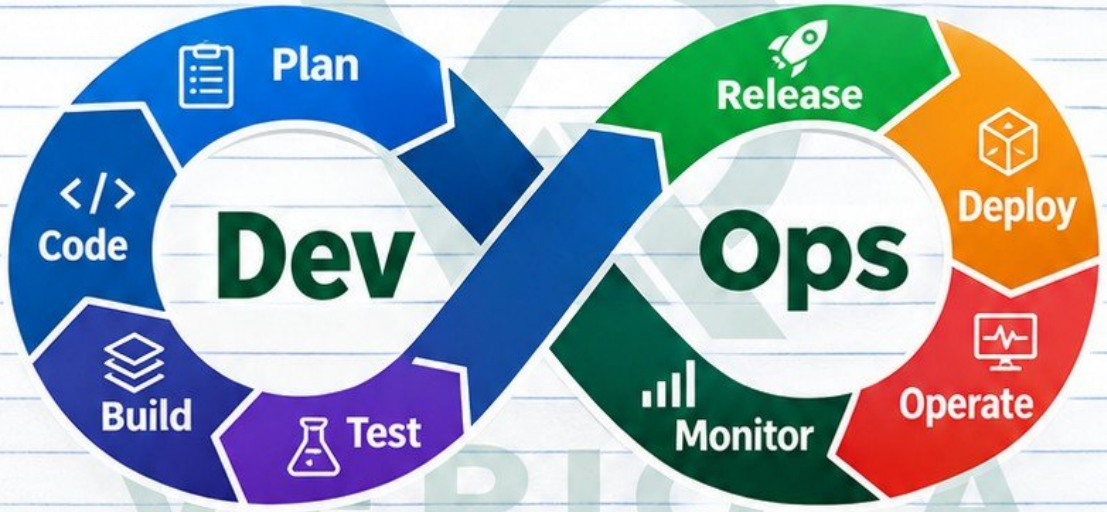
VERIQTA

10 Real-World DevOps Projects

You Can Build and Add to GitHub





✅ Production-Ready | ✅ Real Scenarios | ✅ Hands-On | ✅ Portfolio-Driven

@veriqta Instagram | @veriqta GitHub | @veriqta LinkedIn | @veriqta Telegram | @veriqta X | @veriqta Discord

What Every GitHub Project Must Contain

These are portfolio repositories, not screenshots of completed tutorials. Each project should contain working source code and evidence that another engineer can reproduce and evaluate.

Required Repository Structure

- project-name/
  - README.md
  - architecture/
  - application/
  - infrastructure/
  - automation/
  - policies/
  - tests/
  - observability/
  - incidents/
  - evidence/
  - docs/
    - decisions/
    - runbooks/
    - threat-model.md
  - .github/
    - workflows/

Key Requirements for the README.md

- Explain the business problem.
- Show the solution architecture (with a clear diagram).
- List all prerequisites.
- Provide the exact deployment process (step by step).
- Describe the expected results (with verification).
- Include failure tests (e.g., what you broke and how you tested it).
- Document the recovery process (how to restore service).
- Add cost considerations (estimated cost and optimisation).
- Provide cleanup steps (how to remove all resources).

Note: | Adapt the folders to each project. Do not add empty directories merely to match the template.

Never publish real credentials, sensitive logs, private customer data, or unredacted cloud account identifiers.

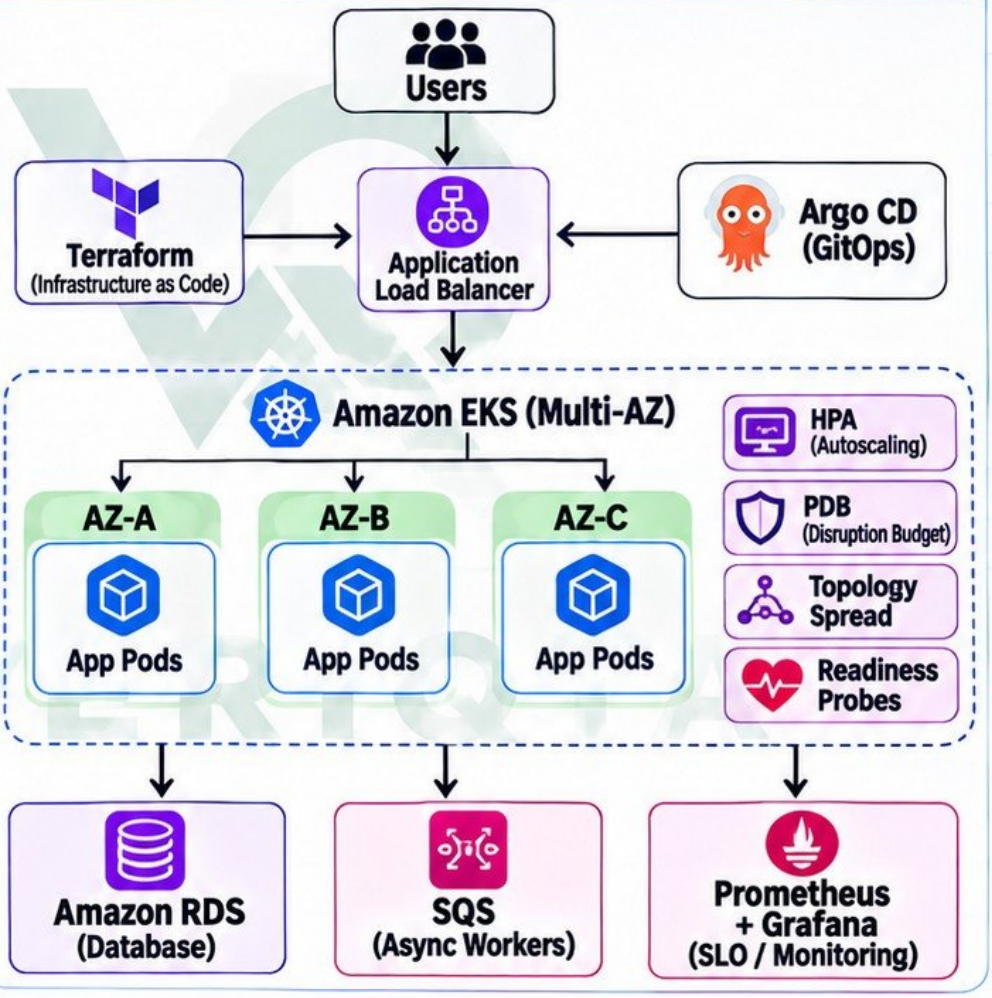
Project 01 | Build a Production Platform That Survives Infrastructure Failure

Business scenario: A SaaS company runs a revenue-generating API on Amazon EKS. The platform must withstand node failures, application crashes, and an Availability Zone outage without unnecessary customer disruption.

Build

- A multi-AZ EKS platform using Terraform and isolated infrastructure state.
- An application with realistic API traffic, database connections, and asynchronous workers.
- Argo CD deployments, topology spread constraints, PodDisruptionBudgets, and autoscaling.
- Readiness checks, SLO dashboards, and controlled workload recovery.
- A recovery controller or automation that distinguishes safe remediation from dangerous repeated restarts.

High-Level Architecture (AWS EKS – Multi-AZ)



Unique engineering challenge: Prevent automated recovery from creating a larger outage when healthy capacity is already exhausted.

Break the Platform and Prove It Can Recover

Test Scenarios (Fault Injection)

- Inject node loss (e.g., terminate a worker node).
- Trigger Pod crashes (e.g., kill or force restart).
- Simulate a zone-level impairment (e.g., AZ outage).
- Create a dependency failure that restarts cannot fix (e.g., database unavailable).

Test, Recover and Validate

Inject Failure (Node / Pod / AZ / Dependency) → Kubernetes Reschedules Workloads → Recovery Controller (with Safety Limits) → Service Recovered (within SLO)

- Prometheus (Metrics)
- Grafana (Dashboards)
- SLO (Error Rate, Latency, Availability)
- Incident Timeline (Before & After)

Execution and Validation

- Observe how Kubernetes reschedules workloads and whether capacity is actually available.
- Introduce safety limits to stop repeated remediation (e.g., restart backoff, failure thresholds).
- Measure detection time, recovery time, error rates, and customer impact.

GitHub Deliverable

Repository:
self-healing-production-platform

Must include:

- [x] Terraform (infrastructure as code)
- [x] Kubernetes manifests (deployments, PDB, autoscaling)
- [x] Recovery automation (e.g., controller/scripts)
- [x] Fault-injection experiments (node, pod, AZ, dependency)
- [x] SLO dashboards (Prometheus + Grafana)
- [x] Incident timelines and before-and-after results

Acceptance Test

Demonstrate recovery within a declared SLO under the tested failures, with no uncontrolled remediation loop.

References / Guidance

- Kubernetes: Topology Spread Constraints (workload distribution and resilience).
- AWS: Resilience Testing Guidance (e.g., AWS Fault Injection Simulator).

VERIQTA

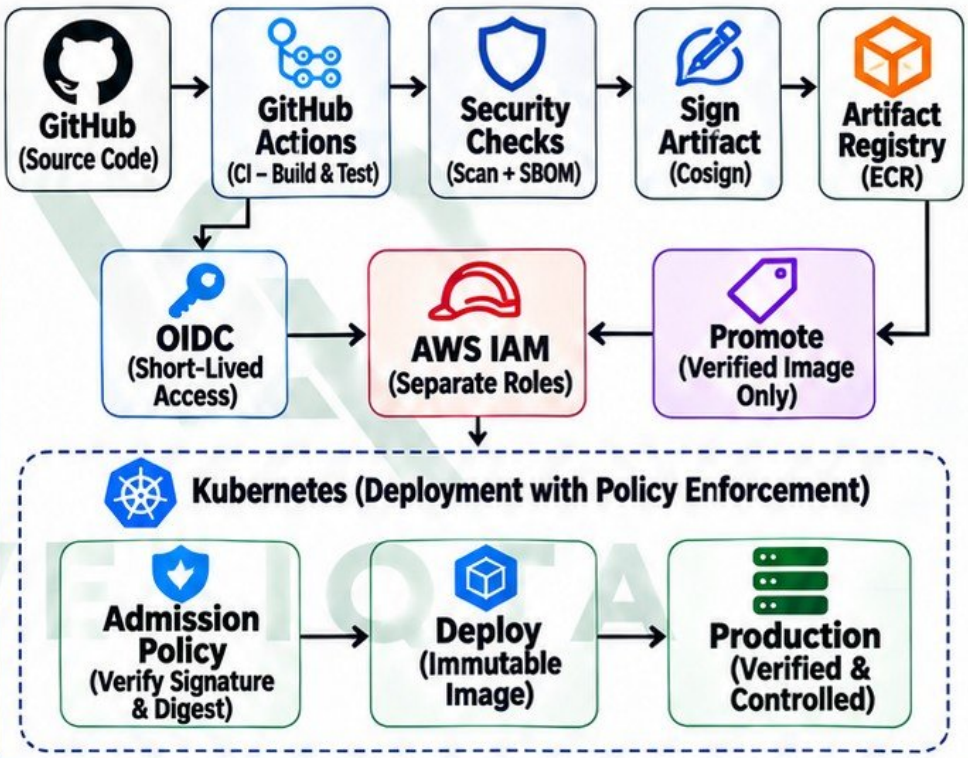
Build a CI/CD System That Does Not Trust Its Own Builds

Business scenario: A financial technology company needs a delivery platform that prevents compromised source code or CI jobs from gaining unrestricted production access.

Build

- Isolated build, verification, and release workflows.
- GitHub Actions authentication to AWS using OIDC and short-lived credentials.
- Separate IAM roles for build and production deployment.
- Software bills of materials (SBOM), vulnerability scanning, and artifact provenance.
- Image signing, digest-pinned deployment, and admission policy enforcement.
- A controlled promotion process that deploys the verified artifact without rebuilding it.

End-to-End Secure Delivery Flow



Unique engineering challenge: Treat otl CI/CD system itself as a potential attacker.

Key Technologies

GitHub Actions | AWS (IAM + OIDC) | Kubernetes (Policies) | ECR (Registry) | Cosign (Signing)

Outcome

- Secure and verifiable delivery pipeline.
- No direct, long-lived production access.
- Only signed, approved artifacts run in production.

Simulate a Software Supply-Chain Compromise

Test the Attack Scenarios

- Introduce an untrusted pull request attempting to access deployment credentials.
- Attempt to deploy an unsigned or unapproved container image.
- Verify that altered artifacts fail provenance or integrity checks.
- Test approval bypass attempts and role-permission boundaries.
- Produce an audit trail connecting commit, build, artifact digest, approval, and deployment.

Supply-Chain Attack Simulation Flow



Security Controls in the Pipeline

OIDC	IAM	SBOM	Artifact Signing	Admission Policy
(Short-Lived Access)	(Least Privilege)	(Dependencies)	(Cosign)	(Verify & Enforce)

GitHub Deliverable

Repository:
zero-trust-delivery-factory

Must include:

- Reusable workflows (build, test, sign, release).
- IAM policies and OIDC configuration.
- Signing verification and admission controls.
- Threat model (attack paths and mitigations).
- Attack-test evidence (logs, failures, and results).

Acceptance Test

Unauthorized code cannot acquire privileged deployment credentials or promote unverified artifacts.

References

- GitHub: OIDC-based authentication and artifact attestations for securing software delivery.

Outcome: A verified, tamper-resistant, zero-trust delivery pipeline that blocks unauthorized code and unverified artifacts.

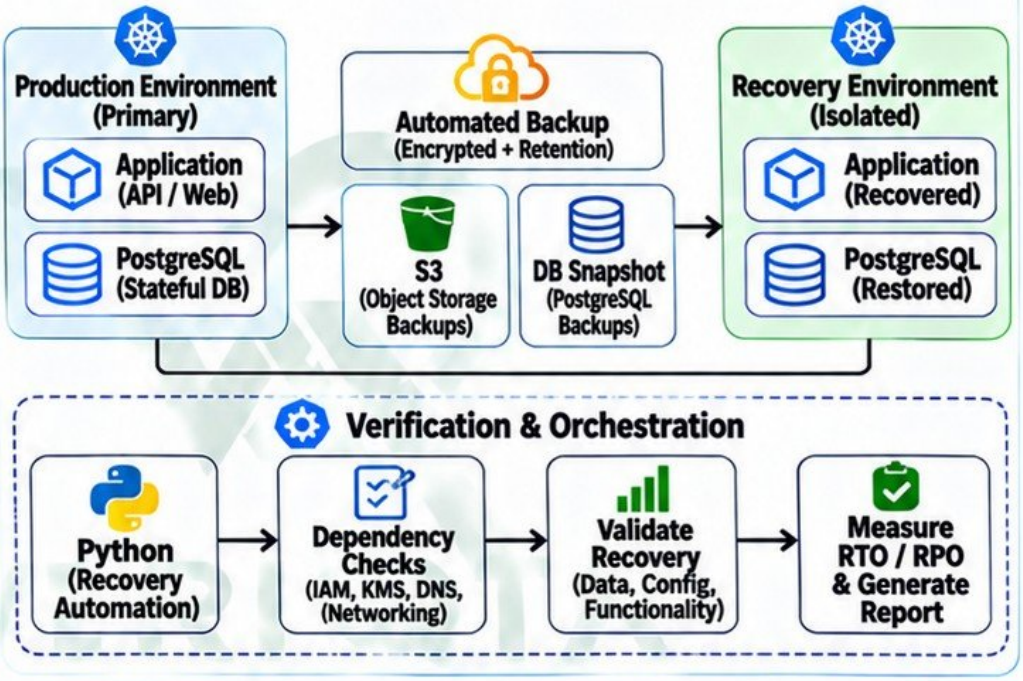
Build an Automated Disaster Recovery Verification Platform

Business scenario: A healthcare technology company reports successful backups every day, but nobody has demonstrated that the business can recover.

Build

- A stateful application with PostgreSQL and object storage.
- Automated encrypted backups with a defined retention policy.
- An isolated recovery environment provisioned through Terraform.
- Python automation to initiate and inspect recovery exercises.
- Dependency checks covering IAM, KMS, DNS, networking, and application configuration.
- RTO and RPO measurements (with evidence).

Disaster Recovery Verification Architecture



Unique engineering challenge: Detect backups that are technically valid but operationally unusable.

GitHub Deliverable

Repository: disaster-recovery-verification-platform

Must include:

- Terraform (production and recovery environments).
- Backup and retention configuration.
- Python automation for recovery exercises.
- Dependency validation scripts (IAM, KMS, DNS, network).
- RTO/RPO measurements and recovery reports.

Outcome

- Automated, testable disaster recovery.
- Verified backups with real recovery evidence.
- Measured RTO and RPO with clear reports.
- Confidence that the business can actually recover.

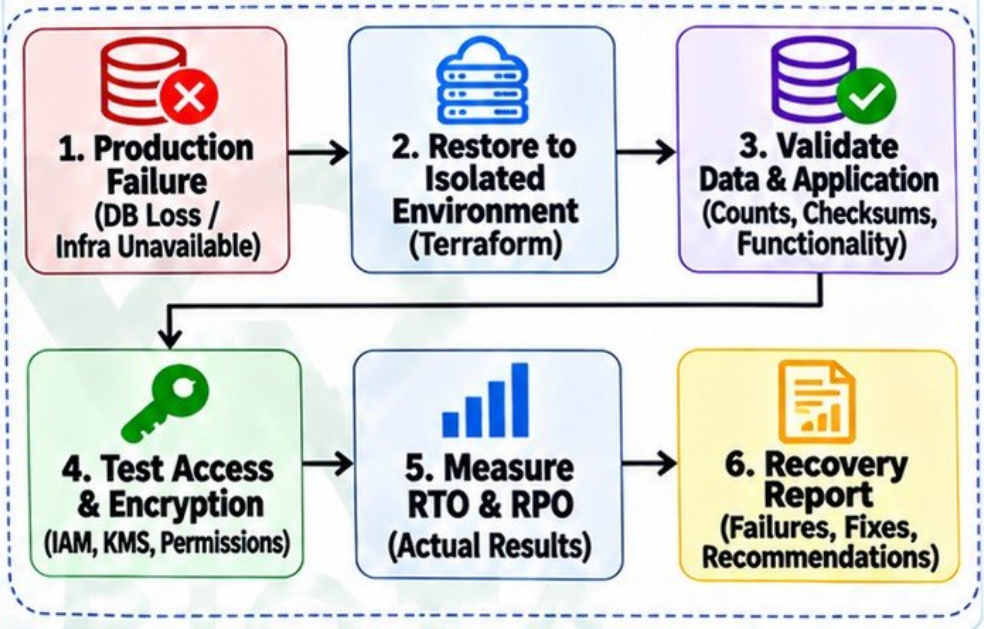
Run a Recovery Exercise Against a Corrupted Production Environment

Scenario: Simulate a production failure (e.g., database loss and unavailable infrastructure), restore the system in a clean, isolated environment, and prove data integrity and business recovery.

Exercise Steps

- Simulate database loss and infrastructure unavailability.
- Restore the application and data into a clean, isolated environment.
- Verify record counts, checksums, application functionality, and data freshness.
- Test for missing permissions and inaccessible encryption keys (e.g., IAM, KMS).
- Produce a recovery report with actual RTO, RPO, failures, and remediation recommendations.

Recovery Exercise Workflow



GitHub Deliverable

Repository:
disaster-recovery-verification-platform

Must include:

- Backup policies and retention configuration.
- Recovery automation (e.g., Terraform and scripts).
- Validation scripts (record counts, checksums, health checks).
- Recovery reports (RTO, RPO, results, and failures).
- Dependency diagrams (infrastructure and application).
- Evidence from successful and failed recovery exercises.

Acceptance Test

A clean recovery environment passes application and data-integrity checks within defined recovery objectives (RTO and RPO).

Outcome

- Proven, repeatable disaster recovery process.
- Verified data integrity, defined RTO/RPO, and actionable remediation.

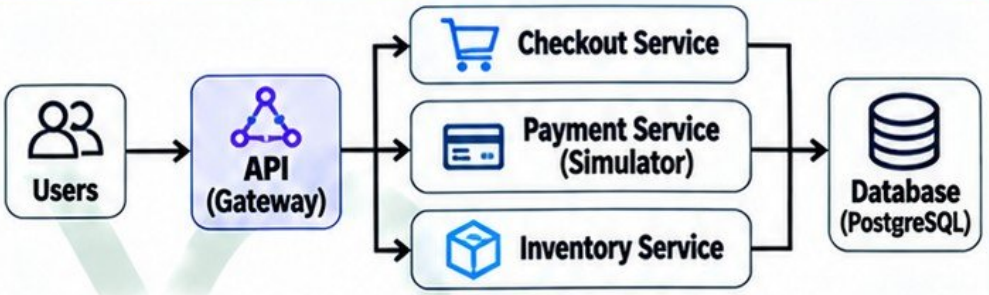
Build an Incident Investigation Platform, Not Just Dashboards

Business scenario: An e-commerce platform experiences intermittent checkout failures. CPU and memory dashboards appear normal, and engineers cannot identify the affected dependency.

Build

- A distributed application with API, checkout, payment simulator, and inventory services.
- OpenTelemetry instrumentation for distributed tracing.
- Metrics, logs, and traces correlated using service identity and trace IDs.
- RED metrics (Rate, Errors, Duration) and saturation indicators.
- Tail-based trace sampling and retention controls.
- SLO burn-rate alerts and an incident investigation dashboard.

Application Architecture



Observability and Investigation Stack



Key Metrics and Investigation

RED Metrics	Request Rate, Error Rate, Request Duration (Latency).
Saturation	Resource and dependency pressure (e.g., DB connections, queue depth, thread pool).
Investigation	Correlate logs, metrics, and traces using trace IDs to find the failing dependency.

Unique Engineering Challenge

Detect high-impact failures that infrastructure utilization metrics do not reveal.

Technologies

OpenTelemetry | Prometheus | Grafana | Loki | Tempo

Outcome

A complete, trace-driven incident investigation platform that quickly identifies the real cause of checkout failures.

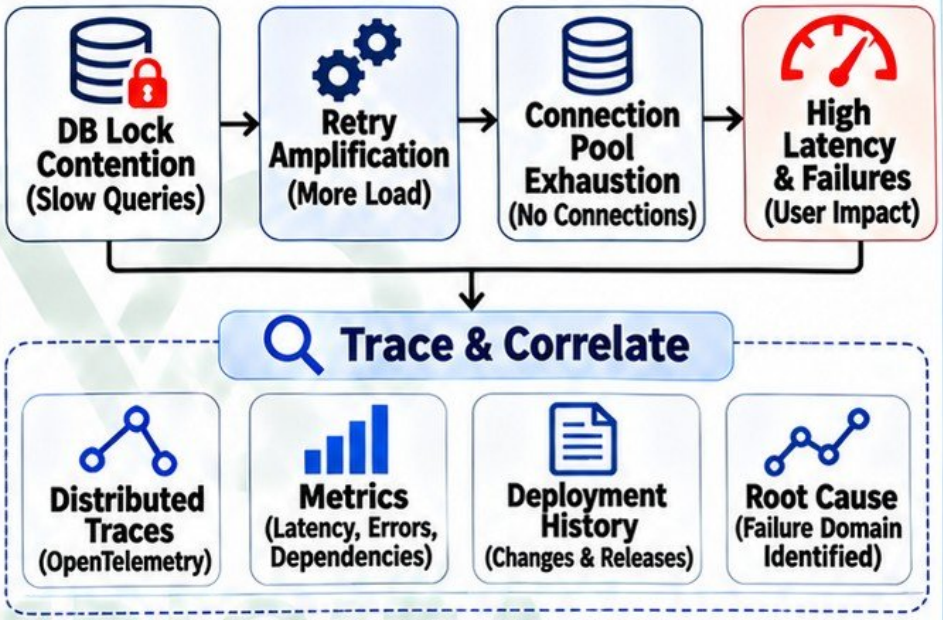
Investigate a Failure Hidden Behind Healthy Infrastructure

Scenario: A production system shows normal infrastructure metrics, but users experience intermittent failures due to a database and dependency issue.

Investigation Steps

- Inject database lock contention and downstream latency.
- Introduce retry amplification and connection pool exhaustion.
- Trace the failing request across service boundaries.
- Correlate symptoms with deployment history and dependency metrics.
- Demonstrate tail sampling for error and high-latency traces.
- Write an evidence-based incident report with corrective actions.

Failure Investigation Flow



GitHub Deliverable

Repository: production-incident-observability-lab

Must include:

- Telemetry configuration (OpenTelemetry).
- Application instrumentation (traces, metrics, logs).
- Dashboards (latency, errors, dependencies, RED).
- Failure scenarios (DB locks, latency, retry, pool exhaustion).
- Trace examples (normal vs. failure).
- Incident report (evidence, root cause, corrective actions).

Acceptance Test

Identify the injected failure domain using correlated telemetry, without relying on prior knowledge of the fault.

Key Technologies

OpenTelemetry | Grafana | Prometheus | Loki | Tempo

Outcome

A complete, evidence-based incident investigation that reveals the real cause behind healthy infrastructure.

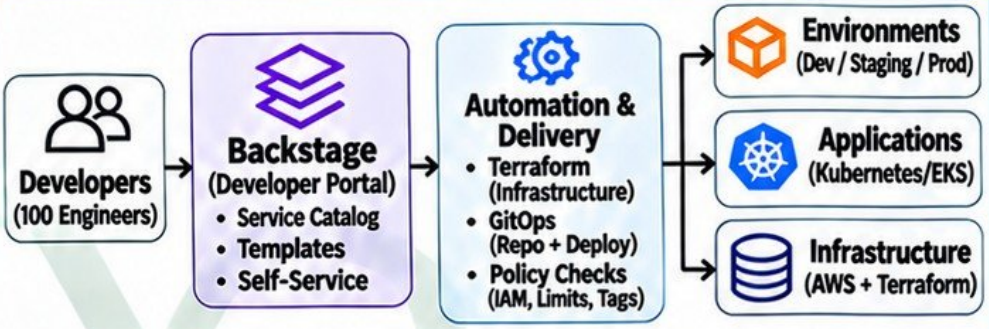
Build a Self-Service Platform for a 100-Engineer Organization

Business scenario: Developers wait several days for DevOps engineers to provision services, configure pipelines, and prepare environments.

Build

- A Backstage developer portal with a software catalog.
- Self-service application provisioning through approved templates.
- Terraform-backed infrastructure workflows.
- Automated GitOps repository creation and deployment configuration.
- Ownership metadata, service documentation, and environment lifecycle management.
- Policy enforcement for IAM, resource limits, tagging, and deployment standards.

Self-Service Platform Architecture



Platform Capabilities

- **Software Catalog** (Services & Ownership)
- **Self-Service Provisioning** (Approved) Templates)
- **GitOps Automation** (Repo Setup & Deploy)
- **Policy Enforcement** (IAM, Limits, Tagging)
- **Lifecycle Management** (Create, Update, Decommission)

GitHub Deliverable

Repository: self-service-platform-backstage

Must include:

- Backstage configuration and plugins.
- Service catalog and provisioning templates.
- Terraform modules (infrastructure as code).
- GitOps setup (repository and deployment configs).
- IAM and policy definitions (least privilege, limits, tagging).
- Documentation (setup, usage, governance, and lifecycle).

Unique Engineering Challenge

Give developers autonomy without giving them unrestricted cloud permissions.

Outcome

- A secure, scalable self-service platform that reduces delivery time from days to minutes.
- Developer autonomy with guardrails, standards, and governance.

Acceptance Test

A developer can provision a new service, deploy it, and access it using approved templates and policies, without manual intervention from a DevOps engineer.

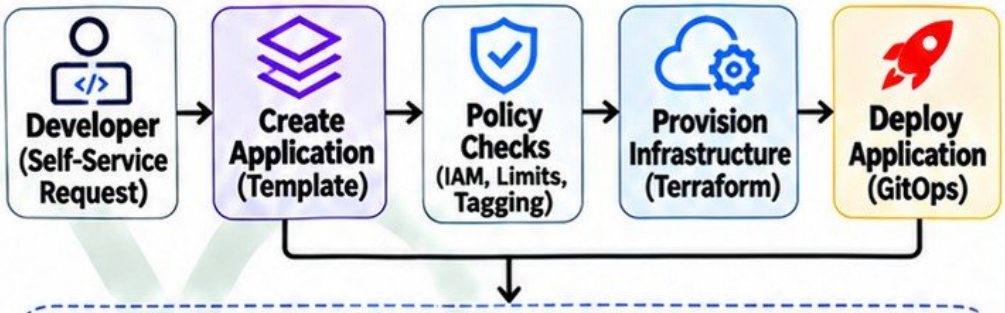
Test Whether the Platform Is Actually Useful

Scenario: Validate the internal developer platform by onboarding applications, enforcing policies, handling failures, and measuring real usage.

Test Steps

- Onboard a new application without platform-team intervention.
- Attempt to request excessive privileges or noncompliant infrastructure.
- Test failed provisioning, duplicate requests, and partial resource creation.
- Implement automatic cleanup for abandoned preview environments.
- Measure onboarding time, failed requests, platform adoption, and support burden.

Platform Evaluation Workflow



Test Cases



GitHub Deliverable

Repository:
enterprise-internal-developer-platform

Must include:

- Portal configuration (e.g., Backstage).
- Service templates and provisioning workflows.
- Policy definitions (IAM, limits, tagging, compliance).
- Terraform infrastructure modules.
- Self-service workflows (create, deploy, cleanup).
- Adoption metrics (onboarding time, success/failure rates, usage, support tickets).

Acceptance Test

A developer can provision and deploy an approved service through the portal without direct infrastructure administrator access.

Outcome

- A functional, policy-enforced self-service platform with measurable business value.
- Faster onboarding, higher adoption, and reduced platform support effort.

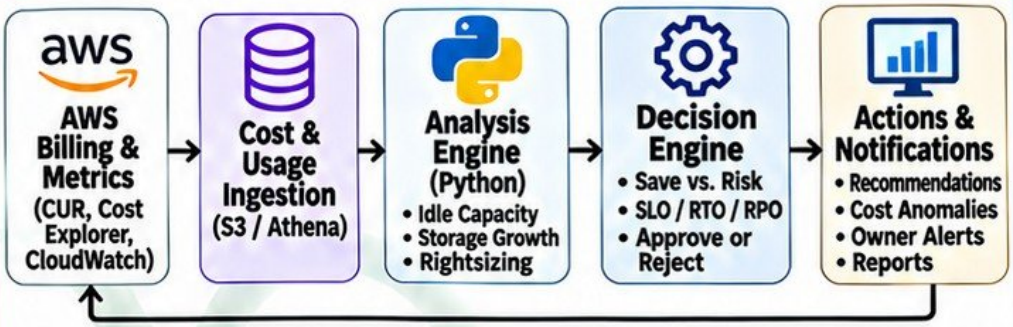
Build an Automated FinOps and Capacity Decision Engine

Business scenario: A SaaS company is overspending on AWS, but aggressive cost-cutting could impact reliability and performance.

Build

- Ingest cost and usage data from AWS billing (e.g., Cost Explorer, CUR).
- Attribute costs to workloads using accounts and tags (e.g., app, env, owner).
- Analyze idle capacity, storage growth, and underutilized resources with Python.
- Recommend rightsizing using CloudWatch performance data (e.g., CPU, memory, network).
- Detect cost anomalies and send ownership-based notifications.
- Evaluate savings against availability, recovery, and capacity requirements.

FinOps Decision Engine Architecture



Key Analyses and Decisions

- **Cost Attribution** (By Account, Tag, Workload)
- **Utilization Analysis** (Idle & Underused)
- **Rightsizing Recommendations** (Performance-Based)
- **Reliability Guardrails** (Reject Savings That Violate SLO, RTO, RPO, or Capacity)

GitHub Deliverable

Repository: finops-capacity-decision-engine

Must include:

- Cost and usage ingestion (AWS Billing/CUR).
- Tagging and workload cost attribution.
- Python analysis and recommendation engine.
- Rightsizing and cost anomaly detection.
- SLO / RTO / RPO guardrail logic.
- Automated reports and ownership notifications.

Unique Engineering Challenge

Reject any cost-saving action that would reduce availability, performance, or recovery capability.

Outcome

- Lower and well-governed cloud spend without compromising reliability.
- Data-driven, production-safe cost recommendations.

Acceptance Test

The engine must recommend cost savings, enforce reliability guardrails, and automatically reject any change that violates defined availability or recovery requirements.

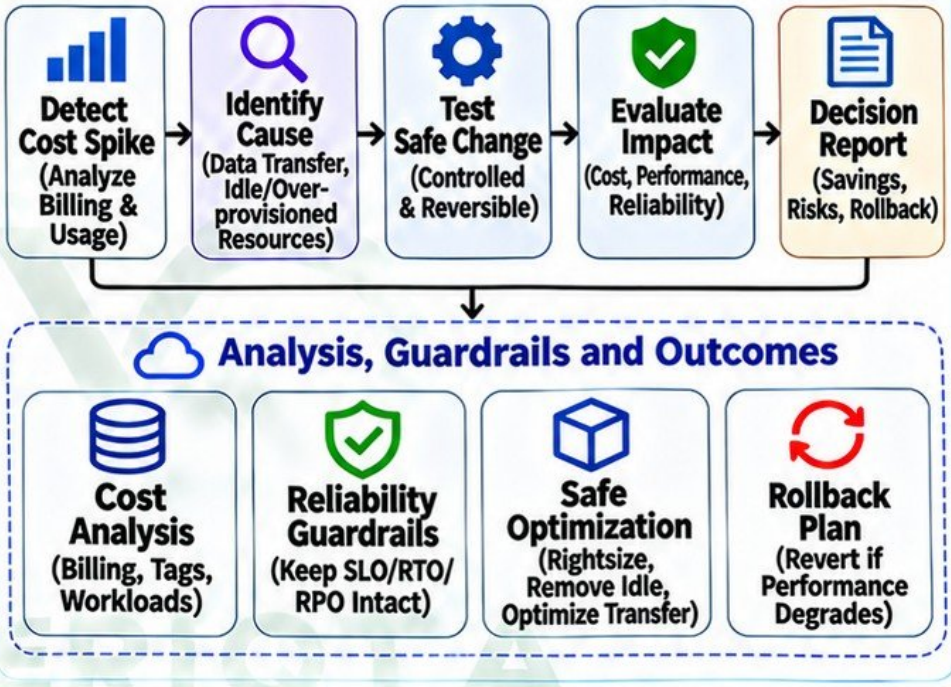
Investigate a Cost Spike Without Breaking Production

Scenario: Investigate an unexpected AWS cost spike, identify the cause, reduce the cost safely, and preserve production reliability.

Investigation Steps

- Simulate a cost spike (e.g., high data transfer and inefficient resources).
- Identify the affected workloads, services, and infrastructure.
- Analyze usage, tagging, and cost breakdown by account, service, and resource.
- Propose and test a controlled configuration change (no production downtime).
- Reject any recommendation that risks critical databases or standby capacity.
- Generate a decision report with expected savings, reliability impact, and rollback plan.

Cost Investigation and Remediation Flow



GitHub Deliverable

Repository:
reliability-aware-finops-engine

Must include:

- Cost analysis scripts (Python).
- Sample anonymized billing data.
- Policy rules and guardrails (SLO, RTO, RPO, capacity).
- Dashboards (cost, usage, anomalies).
- Decision reports (savings, reliability impact, rollback).
- Validation tests (safe vs. unsafe recommendations).

Acceptance Test

The system identifies actionable savings while explicitly preserving declared performance and reliability constraints.

Outcome

- Reduced cloud cost with proven reliability.
- A repeatable, data-driven FinOps process with safety guardrails and rollback capability.

Unique Value

Find and fix the real cost driver, optimize safely, and maintain production stability and recovery readiness.

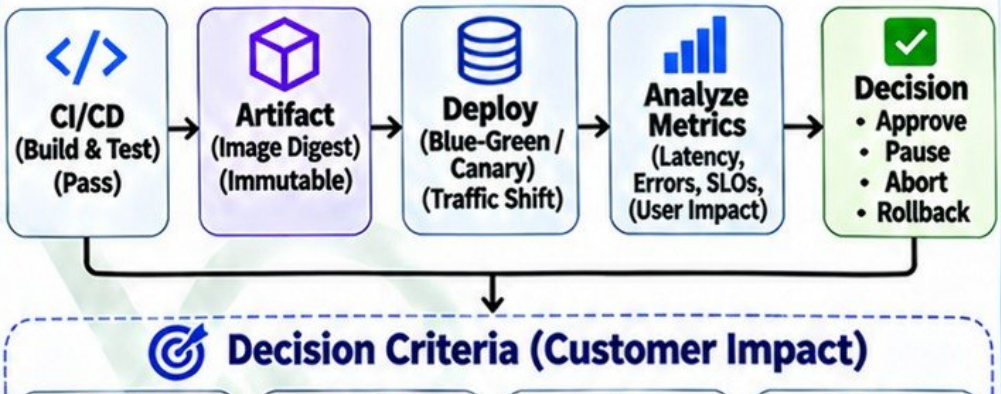
Build an Automated Production Release Decision System

Business scenario: A company deploys frequently, but successful CI/CD runs sometimes introduce customer-facing failures.

Build

- Implement blue-green and canary deployment workflows.
- Use progressive traffic shifting with compatible routing (e.g., ALB, NGINX, Istio).
- Analyze latency, error rate, and application-specific metrics automatically.
- Promote artifacts by immutable image digest.
- Provide release approval, pause, abort, and rollback controls.
- Add database migration compatibility checks (e.g., schema, backward compatibility).

Automated Release Decision Flow



Decision Criteria (Customer Impact)

- **Customer Impact** (Errors, Failed Transactions)
- **Performance** (Latency, Throughput, SLOs)
- **Release Safety** (DB Migration Compatibility, Health Checks)
- **Reliability** (Progressive Shift, Auto-Rollback, Zero Downtime)

GitHub Deliverable

Repository: production-release-decision-system

Must include:

- Deployment workflows (blue-green and canary).
- Traffic shifting and routing configuration.
- Automated metric analysis and decision logic.
- Artifact promotion using image digest.
- Database migration checks and safety gates.
- Approval, pause, abort, and rollback automation.

Unique Engineering Challenge

Make deployment decisions based on real customer impact, not just CI/CD success status.

Outcome

- Safer, automated releases with lower customer impact.
- Faster delivery with controlled risk, validated by real production metrics.

Acceptance Test

The system automatically approves, pauses, aborts, or rolls back a release based on defined customer-impact metrics.

Release a Version That Passes Tests but Fails Under Real Traffic

Scenario: A release works in tests but shows high latency, intermittent checkout errors, or risky database changes in production.

Test and Validate

- Deploy a version with elevated tail latency (e.g., p95/p99 spike).
- Introduce intermittent checkout errors (e.g., random 5xx).
- Validate automated canary analysis and traffic rollback.
- Test a schema change that cannot be safely reversed (e.g., column drop or type change).
- Decide when to roll back application code vs. apply a forward fix (e.g., hotfix or migration).
- Record release decisions, findings, and recovery evidence (metrics, logs, rollout history).

Progressive Delivery and Recovery Flow

Argo Rollouts (Progressive Delivery) → Canary Release (Small (Traffic) → Analyze Live Metrics (Latency, Errors, SLOs) → Decision Promote / Pause / Rollback → Validate Recovery (Service Health & Data Integrity)

Release Decision Guide

- **Tail Latency** (p95 / p99) (Detect Performance Degradation)
- **Checkout Errors** (Intermittent / (User Impact)
- **Schema Risk** (Irreversible Change Detection)
- **Action** Rollback or Forward Fix (Based on Impact & Risk)

GitHub Deliverable

Repository: production-progressive-delivery-engine

Must include:

- Argo Rollouts manifests (canary, rollback, analysis).
- Traffic management configuration (NGINX/ALB/Service).
- Metric queries and analysis templates (latency, errors, SLOs).
- Release policies and decision rules.
- Fault scenarios (latency, intermittent errors, schema change).
- Rollout analysis reports (decisions and recovery evidence).

Acceptance Test

A bad canary is detected and stopped before full promotion, and recovery is verified against service-health criteria.

Outcome

- Safe, automated releases with real-traffic validation.
- Fast detection, automatic rollback, and evidence-based release decisions.

Key Value

Protects customers by using real production signals to approve, stop, or recover from risky releases.

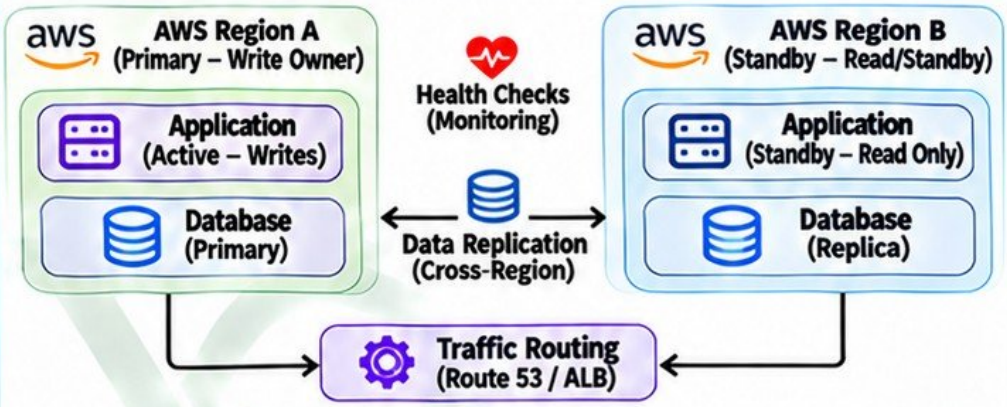
Build a Disaster Recovery Architecture With Controlled Write Ownership

Business scenario: A payment processing platform must survive a regional outage without creating duplicate financial transactions.

Build

- Deploy primary and standby application environments across two AWS Regions.
- Define and implement a data replication and recovery model (e.g., RDS, cross-Region).
- Set up regional health checks and automated traffic-routing controls (e.g., Route 53, ALB).
- Implement an authoritative write-ownership mechanism (e.g., leader/lock, DynamoDB, or DB-based leadership).
- Add application idempotency and transaction reconciliation (e.g., idempotency keys, ledger checks).
- Automate recovery checks with explicit human approval where required (e.g., promotion runbook).

Architecture Overview



Key Controls

- **Write Ownership** (Single Authoritative Region)
- **Idempotency** (Prevent Duplicate Transactions)
- **Reconciliation** (Detect and Resolve Differences)
- **Recovery Approval** (Automated Checks + Human Approval When Required)

GitHub Deliverable

Repository:
disaster-recovery-write-ownership

Must include:

- Multi-Region infrastructure (IaC – Terraform).
- Replication and recovery configuration.
- Write-ownership implementation (e.g., lock/leader).
- Idempotency and reconciliation logic.
- Health checks, routing, and failover automation.
- Recovery runbooks and approval workflow.

Unique Engineering Challenge

Prevent both regions from accepting conflicting writes during an ambiguous failure (e.g., network split or partial outage).

Acceptance Test

- Only one region accepts writes at any time, and no duplicate transactions are created during failover and recovery.
- Recovery is verified through automated checks and required approvals.

Outcome

A resilient, multi-Region payment platform with clear write ownership, controlled failover, and transaction integrity.

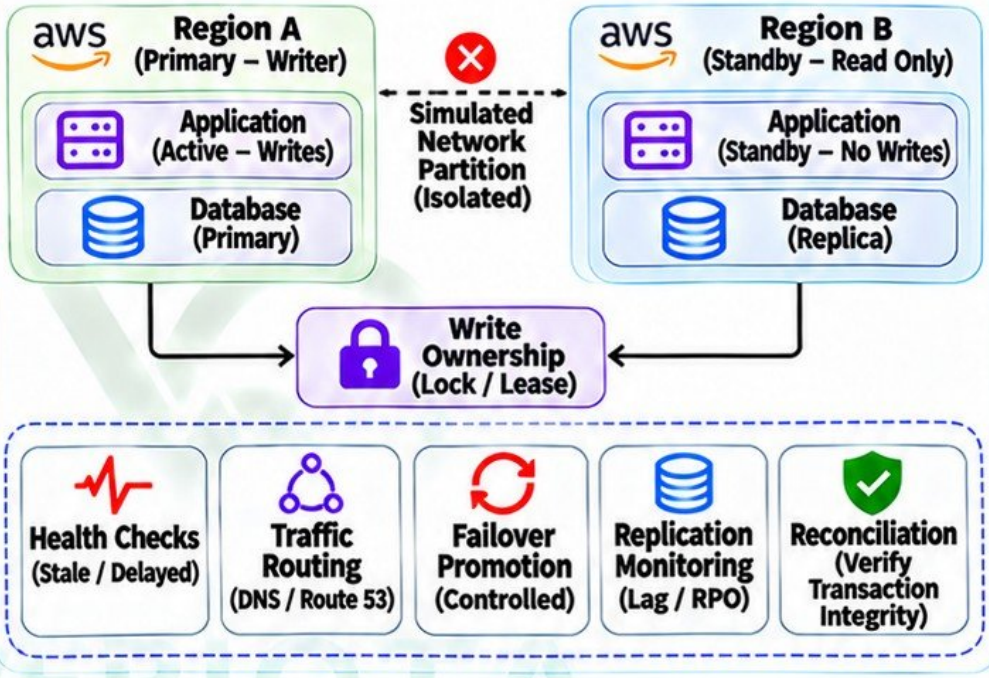
Simulate a Network Partition That Looks Like a Regional Outage

Scenario: Test a network partition to ensure no dual writes, safe failover, and data consistency across regions.

Test and Validate

- Isolate regional communication in a controlled test environment.
- Introduce stale health observations and delayed replication (e.g., network lag).
- Verify the standby cannot become a conflicting writer (write-fencing enforced).
- Test controlled promotion and client reconnection (e.g., DNS / Route 53).
- Measure RTO, RPO, replication lag, and transaction correctness.
- Demonstrate safe restoration of normal operations after recovery.

Test Architecture and Failure Simulation



GitHub Deliverable

Repository:
split-brain-safe-regional-failover

Must include:

- Multi-region Terraform (e.g., AWS).
- Routing controls (e.g., Route 53 / health checks).
- Write-fencing logic (e.g., leader lock / lease).
- Reconciliation and transaction tests.
- Failover and recovery runbooks.
- Incident evidence (logs, metrics, and results).

Acceptance Test

During the tested partition, conflicting write ownership is prevented and transaction integrity is preserved.

Key Metrics

- RTO – Time to restore service.
- RPO – Maximum data loss (target: near zero).
- Replication lag – Measured during the test.
- Transaction correctness – No duplicates or data inconsistencies.

Outcome | A validated, split-brain-safe, multi-region failover with measured RTO, RPO, and verified data integrity.

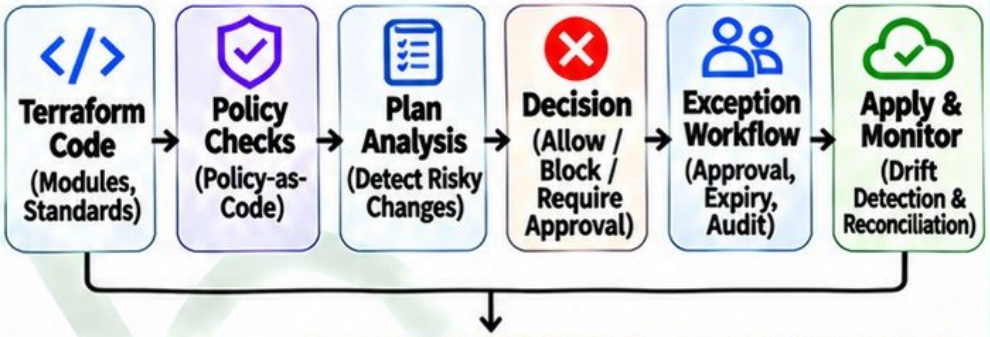
Build an Infrastructure Governance System That Blocks Dangerous Changes

Scenario: A large organization manages hundreds of AWS resources, but inconsistent reviews allow risky changes into production.

Build

- Create reusable Terraform modules with enforceable architecture standards.
- Implement policy-as-code checks against Terraform plans.
- Define rules for public exposure, encryption, IAM scope, resource ownership, and network access.
- Automatically detect risky replacement and deletion plans (e.g., "destroy" or force-recreate).
- Add exception requests with approval, expiration, and audit evidence.
- Implement drift detection and reconciliation workflows (e.g., detect and realign unauthorized changes).

Infrastructure Governance Workflow



Governance Policy Examples



GitHub Deliverable

Repository:
infrastructure-governance-engine

Must include:

- Terraform modules and reference architecture.
- Policy-as-code rules (e.g., OPA / Sentinel).
- CI/CD plan checks and approval workflows.
- Risk detection (replace, delete, force recreate).
- Exception request process (approval and expiry).
- Drift detection and reconciliation jobs.
- Audit logs, reports, and sample change cases.

Unique Engineering Challenge

Detect dangerous infrastructure changes that are valid and deployable, but can cause security, data loss, outage, or compliance risk.

Acceptance Test

- A risky Terraform change (e.g., public exposure or resource deletion) is automatically blocked or requires approved exception.
- Only compliant changes are deployed, and any drift is detected and reconciled.

Outcome

A policy-driven infrastructure governance system that prevents risky changes, ensures compliance, and maintains control at scale.

Simulate a Terraform Change That Could Destroy Production

Scenario: Test governance controls by submitting dangerous infrastructure changes and validating safe rejection, exception handling, and drift reconciliation.

Test and Validate

- Submit a Terraform plan that replaces a stateful database (e.g., RDS).
- Introduce overly permissive IAM policies (e.g., "*:*") and public network exposure (e.g., 0.0.0.0/0).
- Verify that policy-as-code rejects the unsafe changes with clear reasons.
- Test a legitimate emergency exception with approval and expiration (e.g., 24–72 hrs).
- Detect console-created drift (resource changes outside Terraform) and generate a reconciliation plan.
- Verify that policy enforcement does not block approved recovery operations (e.g., restore from backup).

Validation and Control Flow

Terraform Change (Risky Plan) (e.g., DB Replacement) → Policy Checks (OPA / Sentinel) (Evaluate Rules) → Decision (Block / Allow / Exception) (With Reason) → Exception Workflow (Approval, Expiry, Audit) → Drift Detection & Reconcile (Detect, Plan and Apply)

Key Policy Guards

- **Stateful Resource Protection** (Prevent Destructive Replacement)
- **IAM Governance** (Block Overly Permissive Policies)
- **Network Access** (Prevent Unintended Public Exposure)
- **Exception Controls** (Approval + Expiration + Audit Trail)
- **Drift Management** (Detect Changes and Generate Reconciliation)

GitHub Deliverable

Repository: terraform-policy-guardrail-engine

Must include:

- Terraform modules (e.g., RDS, IAM, VPC).
- Policy-as-code rules and tests (OPA / Sentinel).
- Sample plans (dangerous and compliant).
- Drift detection and reconciliation scripts.
- Exception workflow (approval, expiry, and audit).
- Audit reports and validation evidence.

Acceptance Test

Unsafe changes are blocked before application, while legitimate, reviewed exceptions remain possible.

Outcome

- Production is protected from destructive or high-risk changes.
- Governance remains flexible for approved emergency and recovery needs.

Value | A tested, policy-driven Terraform workflow that blocks dangerous changes, manages exceptions, and keeps infrastructure safe.

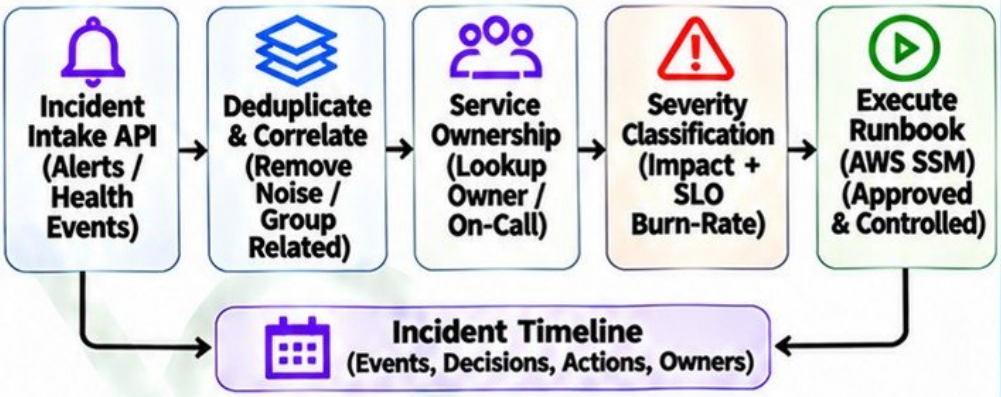
Build an Incident Response System That Coordinates People and Automation

Scenario: A company runs many production services, but during incidents, alerts are noisy, investigations are duplicated, and recovery is uncoordinated.

Build

- Create an incident intake API to receive alerts and service-health events.
- Implement alert deduplication and correlation across multiple sources.
- Map alerts to service ownership and on-call responders.
- Classify severity using service impact and SLO burn-rate evidence.
- Maintain an incident timeline with decisions, actions, and owners.
- Execute approved runbooks through AWS Systems Manager (SSM).
- Verify recovery, log all actions, and generate a post-incident report (with lessons learned).

Incident Response Workflow



Key Components

- **Controlled Automation** (Prevent Duplicate / Conflicting Actions)
- **Recovery Verification** (Validate Service Health & SLOs)
- **Audit Logging** (All Actions, Approvals, and Execution Evidence)
- **Post-Incident Reporting** (Impact, Timeline, Recovery, and Lessons Learned)

GitHub Deliverable

Repository:
incident-response-orchestration-engine

Must include:

- Incident intake API (e.g., Python/FastAPI).
- Alert deduplication and correlation logic.
- Service ownership and on-call configuration.
- Severity and SLO burn-rate evaluation.
- AWS SSM runbooks and automation workflows.
- Incident timeline, audit logs, and reporting.
- Tests and sample incident scenarios.

Unique Engineering Challenge

Ensure automation cannot execute conflicting or repeated recovery actions during a major incident.

Acceptance Test

- A simulated incident is ingested, correlated, classified, and resolved using a controlled runbook without duplicate or conflicting actions.
- Recovery is verified, logged, and a post-incident report is generated.

Outcome

A coordinated, reliable incident response system that detects, classifies, automates, and recovers safely with full accountability.

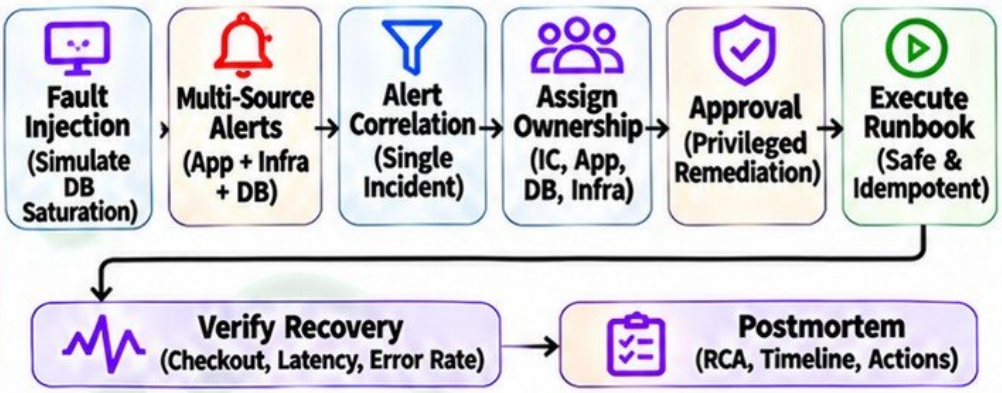
Run a Full Production Incident Simulation

Scenario: Simulate a real production incident where checkout fails due to database saturation, with multiple alerts, coordinated response, safe recovery, and a complete postmortem.

Simulation Steps

- Trigger a checkout outage by simulating database saturation (e.g., high connections / CPU).
- Generate overlapping alerts from application (e.g., high error rate) and infrastructure (e.g., high CPU, queue growth).
- Correlate related alerts into a single incident.
- Assign incident ownership (Incident Commander, DB, App, Infra) and require approval for privileged remediation.
- Execute an approved, idempotent recovery runbook (e.g., scale DB, clear queue, restore capacity).
- Verify customer-facing service recovery with independent health signals (e.g., checkout success rate, latency, error rate).
- Generate a postmortem with root cause, timeline, contributing factors, and preventive actions.

Incident Response Flow (Simulation)



Incident Artefacts and Outputs



GitHub Deliverable

Repository:
production-incident-command-platform

Must include:

- Incident API and service integration.
- Alert correlation rules and deduplication.
- Automation and approved runbooks.
- Audit records and approval workflow.
- Test scenarios (e.g., DB saturation, checkout outage).
- Dashboards (incident status, recovery, SLOs).
- Postmortem examples (RCA, timeline, actions).

AWS Guidance

Test operational procedures using realistic game days and controlled fault injection (e.g., AWS Fault Injection Simulator) instead of relying only on written runbooks.

Acceptance Test

- The system coordinates incident response.
- Duplicate remediation is prevented.
- Recovery is confirmed using independent health signals (e.g., synthetic checkout test, metrics, and logs).

Outcome

A validated, end-to-end incident response system that detects, coordinates, recovers, and documents a real production outage.

Senior Project Portfolio

Project 01 | **The Self-Healing Production Platform That Knows When Not to Heal**

Project 02 | **The Zero-Trust Software Delivery Factory**

Project 03 | **The Disaster Recovery System That Proves Every Backup Works**

Project 04 | **The Observability Platform That Explains Why an Incident Happened**

Project 05 | **The Internal Developer Platform That Eliminates Infrastructure Tickets**

Project 06 | **The Cloud Cost Control System That Protects Reliability**

Project 07 | **The Progressive Delivery System That Stops Bad Releases Automatically**

Project 08 | **The Multi-Region Failover System That Prevents Split-Brain**

Project 09 | **The Policy-Driven Infrastructure Firewall**

Project 10 | **The Production Incident Command and Automated Recovery Platform**